



University of Mianwali

Hangman Game

Submitted by:

Saniah Malik (BCS-F23-M29)

Alina Zahra (BCS-F23-M43)

Program:

BSCS-4th Regular

Instructor:

Ma'am Bakhtawar

Subject:

COAL

Date:

24th June, 2025

Documentation for Hangman Game

1. Introduction

- This project implements a classic Hangman game using x86 Assembly Language on the EMU8086 emulator.
 - The user guesses a hidden word letter-by-letter within a limited number of tries.
 - Features include difficulty levels, hints, high score tracking, and a graphical-style hangman.
-
-

2. Problem Statement

- Creating interactive applications in Assembly Language is complex due to low-level memory management and lack of GUI.
 - This project addresses how to build an engaging game experience within these constraints.
-
-

3. Aim/Objectives

- To implement a fully functional Hangman game in Assembly.
 - To demonstrate the use of loops, conditionals, and memory in low-level programming.
 - To allow gameplay with scoring, hints, and user prompts.
 - To add basic graphical feedback using ASCII art.
-
-

4. Procedure (Step-by-Step Implementation)

- **Step 1:** Initialize data segment and define all game variables.
- **Step 2:** Use system time to randomly select a word and corresponding hint.
- **Step 3:** Allow the user to select difficulty (Easy/Medium/Hard).
- **Step 4:** Clear previous guesses and display welcome banner.
- **Step 5:** Enter the main game loop where the following occurs:
 - Display word progress, hints, score, remaining tries, and hangman figure.
 - Get user input and convert it to uppercase if needed.
 - Check the guess and update game state accordingly.
- **Step 6:** Handle win/loss conditions and update high score if achieved.

- **Step 7:** Ask the user if they want to play again or exit.
-

5. Pros and Cons

- **Pros:**
 - Offers educational value by reinforcing assembly language concepts.
 - Includes interactive feedback with visual ASCII hangman.
 - Customizable words and hints for flexible gameplay.
 - **Cons:**
 - No persistent high score after exit.
 - Text-only interface limits visual richness.
 - Limited input validation (single-letter guesses only).
-

6. Code (Selected Important Functions)

- **Random Word Picker:**
 - Selects a word using system time to provide gameplay variation.

```
random_word:
    mov ah, 2Ch
    int 21h
    mov al, dl
    xor ah, ah
    mov bl, 10
    div bl
```
- **Check Guess Logic:**
 - Compares input with target word, updates score, and modifies tries if incorrect.

```
check_guess:
    mov si, word_ptr
    mov di, offset guessed
    mov cl, word_len
    mov bl, input_char
    xor dx, dx
```
- **Draw Hangman:**
 - Displays the hangman figure based on the number of wrong attempts.

```
draw_hangman:
    mov al, 6
    sub al, tries
    mov bl, al
```
- **Show Guessed Letters:**

- ## 8. Conclusion

- ## 10. Output (Screenshots)

[illegible]

Guessed Letters and Hangman:

```
=====
Enter a letter: f
Word: P _ T _ _ N
Hint: Interpreted language

Guessed letters: D E F K N P T
Tries left: 2
Score: 3
+----+
|      |
| 0     |
|/|\    |
|      |
|      |
+----+

=====
Enter a letter: _
```

Winning or Losing message:

```
=====
Enter a letter: m
*****You Win!*****
```

```
=====
Enter a letter: n
You Lose! The word was: V I R T U A L
? New High Score! ?
Play again? (Y/N): _
```